

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE CODE: CSA14**

**COURSE NAME: COMPILER DESIGN**

---

### COURSE OUTCOME COVERED

**CO3:** *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

*Assessment Tool Weightage: 40%*

---

**QUESTIONS:5**

**Mode: In Class**

**Total Marks:100**

**Hours : 60 mts**

Annexure A

### EXPERIMENTS

---

*I Y ROSHITHA(192424400) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Y ROSHITHA**

***Signature of the student:***

| Q. No                 | Understanding of Concepts (25) | Experimental Design (30) | Use of Tools (20) | Analysis of Results (15) | Documentation (10) | Total Score (out of 100) |
|-----------------------|--------------------------------|--------------------------|-------------------|--------------------------|--------------------|--------------------------|
| 1                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 2                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 3                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 4                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| 5                     | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
| OVERALL RUBRICS VALUE |                                |                          |                   |                          |                    |                          |
|                       | / 4                            | / 4                      | / 4               | / 4                      | / 4                |                          |
|                       | / 20                           | / 20                     | / 25              | / 20                     | / 15               | / 100                    |

## Annexure A

### EXPERIMENTS

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax-directed definitions (SDD). The system must:

- (i) Accept arithmetic expressions with operators +, -, \*, / and parentheses
- (ii) Construct a syntax tree and evaluate expressions using synthesized attributes
- (iii) Respect operator precedence and associativity rules
- (iv) Produce the final computed value of the expression

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Q2.

In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:

- (i) Accept a postfix expression as input
- (ii) Use a stack-based approach to evaluate the expression
- (iii) Demonstrate bottom-up computation of intermediate results
- (iv) Display the final evaluated result

Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.

Q3.

You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:

- (i) Accept arithmetic expressions such as  $a + b * c$
- (ii) Use inherited attributes to pass information during parsing
- (iii) Ensure correct operator precedence
- (iv) Perform evaluation using a top-down approach

Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Q4.

A compiler's semantic analyzer must check type compatibility between expressions. The system should:

- (i) Accept two type expressions such as int, float, int\*, char\*
- (ii) Compare the types for equivalence
- (iii) Handle both basic and pointer types
- (iv) Display whether the types are equivalent or not

Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Q5.

You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:

- (i) Accept operands along with their types
  - (ii) Validate operations such as +, -, , /
  - (iii) *Detect invalid combinations such as int + char*
  - (iv) Report whether the expression is valid or contains a type error
- Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

## RUBRICS FOR CALCULATION

| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax

directed definitions (SDD). The system must:

(i)

Accept arithmetic expressions with operators +, -, \*, / and parentheses

(ii)

Construct a syntax tree and evaluate expressions using synthesized attributes

(iii)

Respect operator precedence and associativity rules

(iv)

Produce the final computed value of the expression

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Software Tool Used : ANTLR 4.13.2

Programming Language Used : Java (JDK 24.0.2)

**CODE:**(Expr. g4)

grammar Expr;

expr returns [int value]

```
: a=term
{
    $value = $a.value;
}
(
    '+' b=term
    {
        $value = $value + $b.value;
    }
    | '-' b=term
    {
        $value = $value - $b.value;
    }
)*
;
```

term returns [int value]

```
: a=factor
```

```

    {
        $value = $a.value;
    }
    (
        '*' b=factor
        {
            $value = $value * $b.value;
        }
        | '/' b=factor
        {
            $value = $value / $b.value;
        }
    )*
;

```

factor returns [int value]

```

: INT
{
    $value = Integer.parseInt($INT.text);
}
| '(' e=expr ')'
{
    $value = $e.value;
}
;

```

INT : [0-9]+ ;

WS : [ \t\r\n]+ -> skip ;

**CODE:**(Main. java)

```

import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;

```

```

public class Main {
    public static void main(String[] args) throws Exception {

        String input = "10+5*2";

        CharStream inputStream = CharStreams.fromString(input);

        ExprLexer lexer = new ExprLexer(inputStream);

        CommonTokenStream tokens = new CommonTokenStream(lexer);

        ExprParser parser = new ExprParser(tokens);

        ParseTree tree = parser.expr();
    }
}

```

```
        System.out.println("Input Expression: " + input);
        System.out.println("Parse Tree: " + tree.toStringTree(parser));
        System.out.println("Final Computed Value: "
            + ((ExprParser.ExprContext) tree).value);
    }
}
```

OUTPUT:

```
C:\antlr>java -cp ".;antlr-4.13.2-complete.jar" Main
Input Expression: 10+5*2
Parse Tree: (expr (term (factor 10)) + (term (factor 5) * (factor 2)))
Final Computed Value: 20

C:\antlr>
```

**Q2.**

**In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:**

- (i) Accept a postfix expression as input**
- (ii) Use a stack-based approach to evaluate the expression**
- (iii) Demonstrate bottom-up computation of intermediate results**
- (iv) Display the final evaluated result**

**Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.**

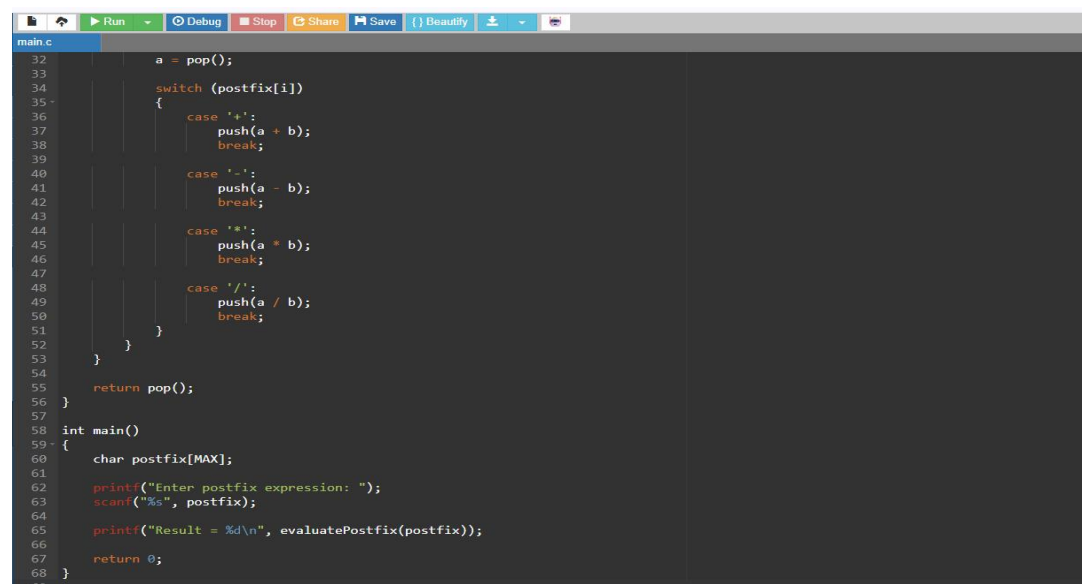
**Software Tool Used : Online GDB**

Programming Language: C

**CODE:**



```
main.c
1 #include <stdio.h>
2 #include <ctype.h>
3
4 #define MAX 100
5
6 int stack[MAX];
7 int top = -1;
8
9 void push(int value)
10 {
11     stack[++top] = value;
12 }
13
14 int pop()
15 {
16     return stack[top--];
17 }
18
19 int evaluatePostfix(char postfix[])
20 {
21     int i, a, b;
22
23     for (i = 0; postfix[i] != '\0'; i++)
24     {
25         if (isdigit(postfix[i]))
26         {
27             push(postfix[i] - '0');
28         }
29         else
30         {
31             b = pop();
32             a = pop();
33
34             switch (postfix[i])
35             {
36                 case '+':
37                     push(a + b);
38                     break;
```



```
39             }
40             case '-':
41                 push(a - b);
42                 break;
43             case '*':
44                 push(a * b);
45                 break;
46             case '/':
47                 push(a / b);
48                 break;
49             }
50         }
51     }
52     return pop();
53 }
54
55 int main()
56 {
57     char postfix[MAX];
58
59     printf("Enter postfix expression: ");
60     scanf("%s", postfix);
61
62     printf("Result = %d\n", evaluatePostfix(postfix));
63
64     return 0;
65 }
```

## OUTPUT:

```
Enter postfix expression: 23*54**9-  
result = 17  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```



**Q3.**

**You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:**

- (i) Accept arithmetic expressions such as  $a + b * c$**
- (ii) Use inherited attributes to pass information during parsing**
- (iii) Ensure correct operator precedence**
- (iv) Perform evaluation using a top-down approach**

**Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing**

**Software Tool Used : Programiz Online C Compiler**

**CODE:**

```
main.c
1 #include <stdio.h>
2
3 int a, b, c;
4
5 int evaluateTerm(int inherited)
6 {
7     int result;
8
9     result = b * c;
10
11     return inherited + result;
12 }
13
14 int evaluateExpression()
15 {
16     int inherited;
17
18     inherited = a;
19
20     return evaluateTerm(inherited);
21 }
22
23 int main()
24 {
25     int result;
26
27     printf("Enter value of a: ");
28     scanf("%d", &a);
29
30     printf("Enter value of b: ");
31     scanf("%d", &b);
32
33     printf("Enter value of c: ");
34     scanf("%d", &c);
35
36     result = evaluateExpression();
37
38     printf("\nExpression: a + b * c\n");
39     printf("Result = %d\n", result);
40
41     return 0;
42 }
43
```

**OUTPUT:**

```
Output
Enter value of a: 10
Enter value of b: 5
Enter value of c: 2

Expression: a + b * c
Result = 20

=== Code Execution Successful ===
```

**Q4.**

**A compiler's semantic analyzer must check type compatibility between expressions. The system should:**

**(i) Accept two type expressions such as int, float, int\*, char\***

**(ii) Compare the types for equivalence**

**(iii) Handle both basic and pointer types**

**(iv) Display whether the types are equivalent or not**

**Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.**

**Software Tool Used :** Google Colab

**Programming Language:** Python

**CODE:**

```
[1] ✓ 34s def check_type_equivalence(type1, type2):  
    if type1 == type2:  
        return True  
    return False  
  
type1 = input("Enter first type: ")  
type2 = input("Enter second type: ")  
  
if check_type_equivalence(type1, type2):  
    print("Types are equivalent.")  
else:  
    print("Types are not equivalent.")
```

**OUTPUT:**

```
#  
v ... Enter first type: int*  
    Enter second type: int*  
    Types are equivalent.
```

**Q5.**

**You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:**

- (i) Accept operands along with their types**
- (ii) Validate operations such as +, -, , /**
- (iii) Detect invalid combinations such as int + char**
- (iv) Report whether the expression is valid or contains a type error**

**Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.**

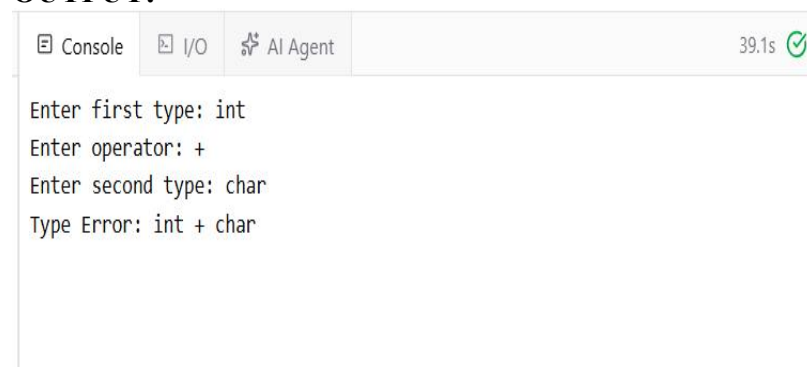
**Software Tool Used : OneCompiler**

**CODE:**



```
1 def type_check(type1, operator, type2):
2     if type1 == "int" and operator == "+" and type2 == "int":
3         return "Valid Expression: int + int"
4
5     elif type1 == "int" and operator == "+" and type2 == "char":
6         return "Type Error: int + char"
7
8     elif type1 == "char" and operator == "+" and type2 == "int":
9         return "Type Error: char + int"
10
11     else:
12         return "Type combination not supported."
13
14
15 type1 = input("Enter first type: ")
16 operator = input("Enter operator: ")
17 type2 = input("Enter second type: ")
18
19 result = type_check(type1, operator, type2)
20
21 print(result)
```

**OUTPUT:**



```
Enter first type: int
Enter operator: +
Enter second type: char
Type Error: int + char
```